CSS

Flex, grid

Theory Notes

1. Flexbox

- **Definition**: A one-dimensional layout system for arranging items in rows or columns.
- **Key properties**:
  - display: flex; → activates flexbox.
  - flex-direction → row (default) or column.
  - justify-content → alignment along main axis (center, space-between, space-around, space-evenly).
  - align-items → alignment along cross axis (center, flex-start, flex-end).
  - flex-wrap → allows items to wrap onto multiple lines.
  - gap → spacing between items.

2. CSS Grid

- **Definition**: A two-dimensional layout system for rows and columns.
- **Key properties**:
  - display: grid; → activates grid.
  - grid-template-columns → defines column sizes.
  - grid-template-rows → defines row sizes.
  - row-gap, column-gap, gap → spacing between cells.
  - justify-items / align-items → alignment inside grid cells.
  - place-content → shorthand for justify-content + align-content.
  - grid-row-start, grid-row-end, grid-column-start, grid-column-end → control item spanning.

3. Display Properties

- block → takes full width, starts on new line.
- inline → flows with text, ignores width/height.
- inline-block → flows inline but respects width/height.
- visibility: hidden → element is invisible but still takes space.
- display: none → element is removed from layout entirely.

Practice Tasks

Short Answer

1. What is the difference between Flexbox and Grid?
2. How does display: none differ from visibility: hidden?
3. When should you use Flexbox instead of Grid?
4. What does flex-wrap do in a flex container?

Coding Exercises

1. Create a Flexbox container with 4 boxes evenly spaced horizontally.
2. Build a responsive navbar using Flexbox.
3. Create a Grid layout with 3 rows and 3 columns, filling each cell with a box.
4. Make one grid item span 2 rows and 2 columns.
5. Demonstrate the difference between block, inline, and inline-block using <p> and .
6. Hide an element using visibility: hidden and another using display: none to compare behavior.
7. Build a simple dashboard layout using CSS Grid.

Flexgrid.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Flex-grid</title>
    <style>
      .container {
        border: 2px solid;
        text-align: center;
        height: 500px;
        display: flex;
        justify-content: center;
        /* justify-content: space-evenly; */
        align-items: center;
        flex-wrap: wrap;
        /* gap: 20px; */
      }
      /* div>p {
        color: red;
      } */

      .container div {
        width: 100px;
        height: 100px;
        border: 2px solid black;
        background-color: burlywood;
        /* margin: 10px ; */
      }
      p {
        /* inline element => width & height ignore  */
        display: inline-block;
        width: 100px;
        height: 100px;
        background-color: yellow;
```

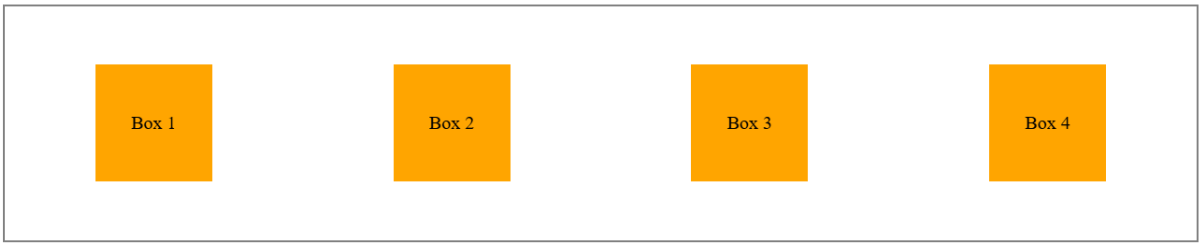
```
      }
      span {
        display: block;
        width: 100px;
        height: 100px;
        background-color: aqua;
      }
      h1{
        visibility: hidden;
      }
    </style>
  </head>
  <body>
    <h1>display none</h1>
    <h2>visibility hidden</h2>
    <p>block element</p>
    <span>inline element</span>
    <h2>inline-block element</h2>
    <h1>Flexbox</h1>
    <div class="container">
      <div>Box1</div>
    </div>
    <!-- <div>
      <p>child element</p>
      <section>
        <p>grand-child element</p>
        <section>
            <p>grand-child</p>
        </section>
      </section>
      <p>child element</p>
    </div> -->

    <!-- <p>block</p>
    <span> inline </span>
    <span>inline</span> -->
  </body>
</html>
```

Grid.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Grid</title>
```

```
  <style>
    .container {
      display: grid;
      /* width: 500px;
       */
      height: 600px;
      border: 2px solid black;
      grid-template-columns: 100px 100px 100px;
      grid-template-rows: 100px 100px 100px;
      /* row-gap: 40px; */
      /* column-gap: 40px; */
      /* gap: 50px; */
      justify-items: center;
      align-items: center;
      place-content: center;
      /* align-content: center; */
      /* justify-content:center; */
      /* align-content: center; */
    }

    .box1{

      grid-row-start: 1;
      grid-row-end: 4;
    }
    .box2{
      grid-column-start: 2;
      grid-column-end: 4;
    }
  </style>
</head>
<body>
  <!-- <table border="2">
    <tr>
      <th rowspan="2">name</th>
      <th>age</th>
      <th>email</th>
    </tr>

    <tr>

      <td>20</td>
      <td>fiit@gmail.com</td>
    </tr>

    <tr>
      <td colspan="3">ganesh</td>
    </tr>
```

```
    </table> -->
    <h1>Grid element</h1>

    <div class="container">
      <div class="box1">Box1</div>
      <div class="box2">Box2</div>
      <div>Box3</div>
      <div>Box4</div>

    </div>
  </body>
</html>
```



Grid Example